

Report - Practical 06

Genetic Algorithms for 0/1 Knapsack

SE3062 - Intelligent Systems

Y3S1 - 2026

Student Name: Sanjeevsaran J.
Student ID: IT24104155

Practical 06: Genetic Algorithms for 0/1 Knapsack

SE3062 – Intelligent Systems | Experimental report

1. Problem and objective

The 0/1 Knapsack Problem is about choosing which items to pack so that the total value is as high as possible without going over the weight limit. An item is either chosen (1) or left out (0). I used a Genetic Algorithm (GA) to search for a good combination and compared how changing the operators affects its performance.

2. Dataset and GA design

For a reproducible demonstration, I generated 100 items using NumPy random seed 2026. Item weights were sampled as integers from 1 to 30, and item values from 10 to 100. The sum of all weights is 1,538, and the knapsack capacity was set to 538 (35% of the total weight, rounded down). This is a synthetic test instance, not an OR-Library or Pisinger instance.

Each chromosome contains 100 binary genes. I calculated its value and weight by adding up the items where the gene is 1. To discourage solutions that exceed the capacity, I used a hard penalty: fitness = total value − 1,000 × max(0, total weight − 538). The penalty is strong enough that overweight solutions are very unlikely to be preferred.

Baseline configuration

Setting	Value	Setting	Value
Population	150	Generations	200
Selection	Tournament, k = 3	Crossover	Two-point, p = 0.90
Mutation	Bit flip, p = 0.02 per gene	Elitism	Best 2 preserved
Repeated runs	Seeds 42, 43, 44	Fitness	Hard penalty

3. Experiment method

I first ran the baseline and recorded the best fitness and average fitness across the generations. I then changed only one setting at a time: tournament size to 2, crossover to uniform, or mutation probability to 0.10. I used the same dataset, population size, generation limit, crossover probability, elitism, and three random seeds for every variant to keep the comparison fair.

4. Results

Table 1 shows the final best fitness after 200 generations. The values are measured results from the accompanying Python experiment, not estimates.

Variant	Best fitness (mean ± SD)	Reached baseline median (of 3 runs)	Observation
Baseline (k=3, 2-point, pm=.02)	3374.7 ± 12.0	2 / 3	Strongest average in this test
Tournament k=2	3264.3 ± 9.1	0 / 3	Lower selection pressure
Uniform crossover	3365.0 ± 23.6	1 / 3	Similar result, more variable
Mutation pm=0.10	2935.7 ± 77.5	0 / 3	High mutation disrupted progress

The baseline median final fitness was 3,374. For this comparison, “reached baseline median” means the run reached a fitness of at least 3,374 at some point in its 200 generations. For the baseline, two runs did so at generations 198 and 178. Uniform crossover reached it in one run at generation 121.

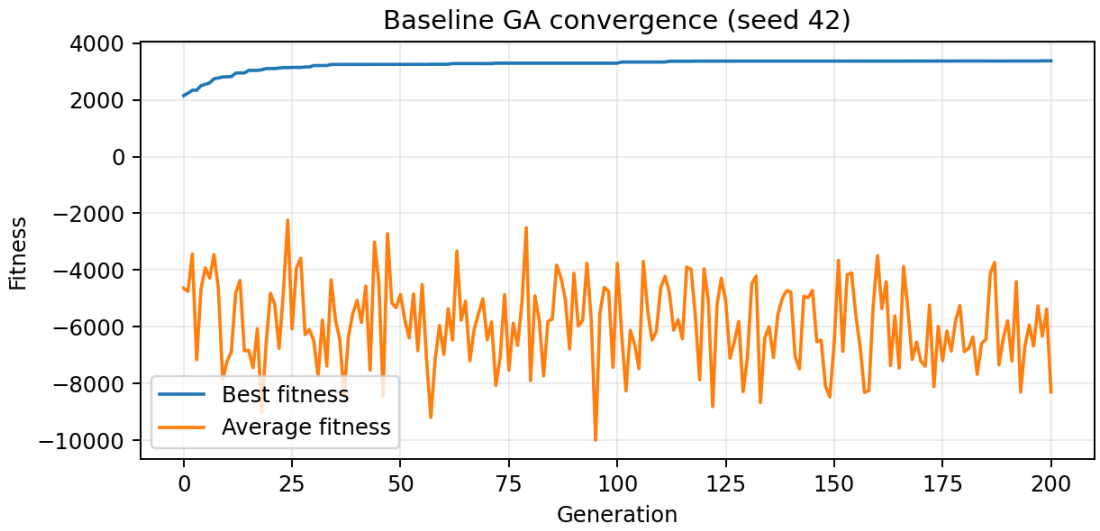


Figure 1. Baseline best and average fitness over generations (seed 42).

5. Discussion

In my tests, the baseline gave the highest average best fitness. Reducing the tournament size from 3 to 2 made selection less competitive, but it also produced a lower final fitness within the same number of generations. This suggests that a little more selection pressure was useful for this dataset.

Uniform crossover performed quite close to the baseline, although the results varied more between seeds. It mixes genes more freely, which may help exploration but can also break a useful combination of selected items. Increasing mutation to 0.10 gave clearly worse results. Too many random bit changes made it harder to keep improving good solutions, even though the two best individuals were preserved.

The fitness graph shows that the best solution generally improves over time and then levels off. Average fitness is less smooth because crossover and mutation keep introducing weaker individuals. This is normal for a GA and does not necessarily mean the best solution is getting worse.

6. Conclusion

This practical helped me understand how selection, crossover, mutation and elitism work together in a Genetic Algorithm. For this 100-item test, tournament selection with k = 3, two-point crossover, mutation probability 0.02 and elitism of 2 gave the best average result among the settings I tested. I would use these settings as a starting point, while still trying more seeds and a standard benchmark before calling them the best settings in general.

Reproducibility and scope

Experiment script: practical06_experiment.py. Results: practical06_results.csv. Dataset seed: 2026; run seeds: 42, 43, 44. NumPy was used for the GA implementation. The lab sheet also asks for a standard OR-Library or Pisinger instance and the use of DEAP, PyGAD or pymoo; this demonstration uses a synthetic instance and a direct NumPy implementation, so it does not yet meet those two specific requirements.